End Term Project - 1 Report

On

Practical Programming with C (CSE 3544)

BUILDING A SIMPLE SHELL

B. Tech. CSE 5th Semester
Submission:
Section: 23412K1

Subject: Practical Programming with C (CSE 3544)

Submitted by:

Name: Amit Kumar

Regd. No.: 2341011129

Guided by: Ishan Ayus
Academic Year: 2025 - 2026
Department: CSE

INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH

(FACULTY OF ENGINEERING)

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY), BHUBANESWAR, ODISHA

Abstract

The objective of this project is to design and implement a simplified command-line shell in the C programming language that replicates the core functionalities of a Unix-based shell by acting as an interface between the user and the operating system, interpreting user commands and executing them using system-level calls. The implementation includes execution of simple commands, support for command-line arguments, process creation using fork(), program execution using execvp(), inter-process communication through pipe(), and input-output redirection using file descriptor manipulation functions such as open(), dup2(), and close(). Developed in a Linux environment using standard POSIX system calls, the shell continuously displays a prompt, reads user input, tokenizes the command string, detects special operators such as pipe (|), input redirection (<), and output redirection (>), and executes the appropriate logic. For simple commands, a child process is created and replaced using execvp(), while piped commands involve multiple processes connected through properly configured pipes to enable cascading data flow, and redirection is handled by duplicating file descriptors to standard input or output streams. The implemented shell successfully executes standalone commands, commands with arguments, piped and cascading piped commands, and supports input and output redirection, demonstrating correct process synchronization, file descriptor management, and behavior consistent with a basic Unix shell.

Submitted By: Amit Kumar

Guided By: Ishan Ayus

Table Of Contents

List Of Figures

1. Introduction

A shell is a command-line interpreter that provides a textual interface between the user and the operating system. In Unix and Linux systems, the shell plays a critical role in interpreting user commands, initiating program execution, and managing system-level operations through system calls. Unlike graphical interfaces, command-line shells offer greater control, flexibility, and efficiency for interacting with system resources. They enable users to execute programs, manipulate files, perform process control, and utilize inter-process communication mechanisms.

At its core, a shell reads input commands from the user, parses them into executable components, and creates child processes to execute the requested operations. This involves fundamental operating system concepts such as process creation, program replacement, file descriptor management, and synchronization. System calls like fork() are used to create new processes, while functions from the exec() family replace the child process image with the specified program. The parent process waits for the child to complete execution using wait() or waitpid(), ensuring proper synchronization and resource management.

Modern shells also support advanced functionalities such as command-line arguments, pipelines, and input-output redirection. Piping allows the output of one command to serve as the input to another, implemented through the pipe() system call and file descriptor duplication using dup2(). Redirection enables commands to read from or write to files instead of standard input and output streams, enhancing flexibility and automation capabilities.

The objective of this project is to design and implement a simplified Unix-like shell in the C programming language that supports execution of simple commands, commands with arguments, multiple piped commands, and input-output redirection. The implementation demonstrates practical understanding of process management, inter-process communication, and file descriptor handling in a Linux environment. This project provides hands-on exposure to core operating system principles and system-level programming techniques essential for advanced software development and systems engineering.

2. Problem Statement

The objective of this project is to design and implement a simplified command-line shell in the C programming language that emulates the core functionalities of a Unix/Linux shell. The system must accept user commands interactively, interpret them correctly, and execute them using appropriate system calls. The shell should support execution of simple commands, commands with arguments, multiple piped commands, and input-output redirection. The implementation must use low-level POSIX system calls such as fork(), execvp(), pipe(), open(), dup2(), and waitpid() to demonstrate understanding of operating system process management and inter-process communication. The challenge lies in correctly parsing user input, managing multiple child processes, handling file descriptors efficiently, and ensuring proper synchronization between processes while maintaining continuous interactive operation.

3. Objectives of the Project

The primary objective of this project is to gain practical exposure to system-level programming concepts in a Unix/Linux environment. The project aims to implement a working shell that can execute external commands and manage multiple processes dynamically. Specific objectives include understanding and implementing process creation using fork(), executing programs using execvp(), synchronizing parent and child processes using waitpid(), and enabling inter-process communication using pipe(). Another objective is to implement input and output redirection through file descriptor manipulation using open() and dup2(). Additionally, the project aims to strengthen knowledge of command parsing, tokenization, memory management, and modular program design in C. Through this implementation, the student develops a strong conceptual and practical understanding of operating system internals.

4. Tools & Technologies Used

The project was developed using the C programming language, which provides direct access to system-level operations through standard POSIX libraries. The implementation relies heavily on system calls such as fork(), execvp(), pipe(), dup2(), open(), close(), and waitpid() for process and file descriptor management. The GNU Compiler Collection (GCC) was used for compilation of the program. Standard header files including stdio.h, stdlib.h, string.h, unistd.h, sys/types.h, sys/wait.h, and fcntl.h were utilized. The development and testing were performed in a Linux terminal environment, ensuring compatibility with Unix-based process handling mechanisms.

Development Environment

The development was carried out in a Linux-based environment using the GCC compiler. The program was written and compiled through the terminal interface, which allowed direct testing of shell functionalities. The interactive environment enabled verification of command execution, pipe operations, and redirection mechanisms in real time. Debugging was performed using compiler warnings and systematic testing of different command combinations. The modular structure of the program facilitated easier debugging and testing of individual components such as parsing, redirection handling, and pipe execution.

Operating System Used

The project was implemented and tested in a Linux operating system environment. Linux provides a POSIX-compliant system architecture, which supports system calls required for process management and inter-process communication. The availability of standard Unix utilities such as ls, who, grep, wc, and cat enabled effective testing of the implemented shell. The operating system environment ensured proper execution of forked processes, file descriptor duplication, and pipe-based communication between processes.

5. Methodology

The shell operates in a continuous loop where it displays a prompt, reads user input, tokenizes the command, and analyzes it for pipes or redirection operators. If no special operators are detected, a child process is created using fork() and the command is executed using execvp(). For redirection, the required file is opened and duplicated to standard input or output using dup2() before execution. For piped commands, multiple processes are created and connected through pipe() to enable data flow between them. After execution, the parent process waits for all child processes to terminate and resumes prompt display.

Understanding the Problem

The key challenge in implementing a shell lies in proper process management and file descriptor handling. Each command must execute in a separate child process while the main shell continues running. Piped commands require concurrent execution with correctly connected input and output streams. Redirection involves replacing standard input or output descriptors with file descriptors. Proper closure of unused descriptors is necessary to prevent resource leakage and ensure correct execution flow.

Program Development

The program is divided into modular functions for prompt display, input reading, command parsing, pipe detection, redirection handling, and execution. Tokenization is performed using strtok(). Piped commands are separated into individual command arrays and connected using pipes. Redirection handling identifies file names and adjusts file descriptors before execution. This modular structure improves readability and simplifies debugging.

Execution

The shell successfully executes simple commands, commands with arguments, piped commands, cascading pipelines, and input-output redirection. Standard utilities such as ls, who, and date execute correctly. Multi-stage pipelines and redirection commands function as expected, maintaining behavior consistent with a basic Unix shell.

Testing & Debugging

Testing covered simple execution, argument handling, single and multiple pipes, and redirection scenarios. Edge cases such as empty input and invalid commands were verified. Debugging focused on correct process synchronization and file descriptor management. Compilation warnings were resolved to ensure stable performance.

Result & Analysis

```
[amit07]$ ls
cat cat.c myinput.txt myshell output.txt proj2.c sample.txt test1.txt test2.txt
[amit07]$ who
amit07 pts/1
[amit07]$ date
2025-12-27 10:46
Sat Dec 27 12:24:44 UTC 2025
[amit07]$ pwd
/home/amit07/ppwc
```

Figure 1: Execution of simple commands (Part 1)

```
[amit07]$ ls -l
total 72
-rwxr-xr-x 1 amit07 amit07 16592 Dec 27 11:53 cat
-rw-r--r-- 1 amit07 amit07  4012 Dec 27 11:52 cat.c
-rw-r--r-- 1 amit07 amit07    39 Dec 27 12:24 myinput.txt
-rwxr-xr-x 1 amit07 amit07 17184 Dec 27 12:23 myshell
-rw-r----  1 amit07 amit07    54 Dec 27 12:04 output.txt
-rwxr-xr-x 1 amit07 amit07  5984 Dec 27 12:20 proj2.c
-rw-r--r-- 1 amit07 amit07    23 Dec 27 12:24 sample.txt
-rw-r--r-- 1 amit07 amit07    68 Dec 27 11:53 test1.txt
-r-r----   1 amit07 amit07    40 Dec 27 11:53 test2.txt

[amit07]$ ls -la
total 80
drwxr-xr-x 2 amit07 amit07  4096 Dec 27 11:24 .
drwxr-x--- 8 amit07 amit07  4096 Dec 27 11:50 ..
-rwxr-xr-x 1 amit07 amit07 16592 Dec 27 11:53 cat
-rw-r--r-- 1 amit07 amit07  4012 Dec 27 11:52 cat.c
-rw-r----  1 amit07 amit07    39 Dec 27 12:24 myinput.txt
-rwxr-xr-x 1 amit07 amit07 17184 Dec 27 12:23 myshell
-rw-r----  1 amit07 amit07    54 Dec 27 12:04 output.txt
-rwxr-xr-x 1 amit07 amit07  5984 Dec 27 12:20 proj2.c
-rw-r----  1 amit07 amit07    23 Dec 27 12:24 sample.txt
-rw-r----  1 amit07 amit07    68 Dec 27 11:53 test1.txt
-rw-r----  1 amit07 amit07    40 Dec 27 11:53 test2.txt
[amit07]$ ls -l /usr/local
total 32
drwxr-xr-x 2 root root 4096 Aug 5 16:55 bin
drwxr-xr-x 2 root root 4096 Aug 5 16:55 etc
drwxr-xr-x 2 root root 4096 Aug 5 16:55 games
drwxr-xr-x 2 root root 4096 Aug 5 16:55 include
drwxr-xr-x 3 root root 4096 Aug 5 16:55 lib
lrwxrwxrwx 1 root root    9 Aug 5 16:55 man -> share/man
drwxr-xr-x 2 root root 4096 Aug 5 16:55 sbin
drwxr-xr-x 7 root root 4096 Aug 5 16:57 share
drwxr-xr-x 2 root root 4096 Aug 5 16:55 src
```

Figure 2: Command execution with arguments (Part 2)

```
[amit07]$ who | grep amit07
amit07 pts/1        2025-12-27 10:46
[amit07]$ ls | wc -l
9
[amit07]$ ls -l | grep shell | wc -l
1
[amit07]$ cat sample.txt | grep Line | wc -l
3
```

Figure 3: Pipe and cascading pipe commands (Part 3)

```
[amit07]$ echo "This is test content for redirection" > myinput.txt
[amit07]$ echo -e "Line 1\nLine 2\nLine 3" > sample.txt
[amit07]$ cat > output.txt
Hello from my shell project[amit07]$
[amit07]$ cat < output.txt
Hello from my shell project [amit07]$ cat sample.txt > copy.txt
[amit07]$ cat copy.txt
"Line 1
Line 2
Line 3"
[amit07]$ cat < myinput.txt > myoutput.txt
[amit07]$ cat myoutput.txt
"This is test content for redirection"
[amit07]$
```

Figure 4: Input-output redirection (Part 4)

Logical Understanding

The implemented shell is based on the fundamental parent–child process model of Unix systems. The main shell process runs in an infinite loop, continuously accepting user input and delegating execution responsibilities to child processes. When a command is entered, the shell parses the input and determines whether it is a simple command, a piped command, or a redirection-based command. For execution, the fork() system call creates a child process, and execvp() replaces the child's process image with the requested program.

For piped commands, multiple child processes are created, and pipe() establishes communication channels between them. The dup2() function is used to redirect standard input and output to appropriate pipe ends, enabling sequential data transfer between processes. In redirection scenarios, file descriptors associated with files are duplicated to standard input or output streams before execution. The parent process uses waitpid() to ensure synchronization and prevent zombie processes. This structured coordination of parsing, process creation, descriptor management, and synchronization forms the logical backbone of the shell implementation.

Conclusion

The project successfully implements a simplified Unix-like shell using the C programming language and standard POSIX system calls. The developed shell is capable of executing simple commands, handling command-line arguments, supporting single and multiple piped commands, and performing input-output redirection. Through the use of system calls such as fork(), execvp(), pipe(), open(), dup2(), and waitpid(), the project demonstrates practical application of process creation, program execution, file descriptor manipulation, and inter-process communication mechanisms in a Linux environment.

The implementation ensures proper synchronization between parent and child processes while maintaining continuous interactive operation. Careful handling of file descriptors prevents resource leakage and guarantees correct data flow in both redirection and pipeline scenarios. The modular design of the program enhances clarity, maintainability, and logical organization.

Overall, the project provides strong practical exposure to operating system concepts and system-level programming techniques. It reinforces understanding of process management and communication in Unix-based systems and establishes a solid foundation for advanced topics in systems programming, operating system design, and software development.

Future Scope & Applications

The current implementation provides fundamental shell functionality; however, it can be extended significantly. One major enhancement would be the implementation of built-in commands such as cd, exit, and pwd, which require execution within the parent process. Support for background execution using the & operator and implementation of job control mechanisms would further improve usability. Signal handling (e.g., handling Ctrl+C) could also be integrated to manage process interruption gracefully.

Additional improvements may include command history tracking, auto-completion features, environment variable expansion, and improved error reporting. Memory optimization and dynamic argument handling can enhance scalability.

The knowledge gained from this project is applicable in operating system development, embedded systems programming, system utility design, and advanced Unix/Linux system administration tools. It forms a foundation for understanding kernel-level process scheduling and system resource management.

References

1. Harwani, B. M., *Practical C Programming*. Birmingham, Mumbai: Packt Publishing, 2020.
2. Hanly, J. R., and E. B. Koffman, *Problem Solving and Program Design in C*, 8th ed., Global Edition. Essex: Pearson Education, 2016.
3. Robbins, K. A., and S. Robbins, *UNIX Systems Programming*. Upper Saddle River, NJ: Prentice Hall PTR, 2003.
4. Stevens, W. Richard, and Stephen A. Rago, *Advanced Programming in the UNIX Environment*. Addison-Wesley.
5. Silberschatz, A., P. B. Galvin, and G. Gagne, *Operating System Concepts*. Wiley Publications.
6. Linux Manual Pages – fork(2), execvp(3), pipe(2), dup2(2), open(2), waitpid(2).
7. IEEE POSIX System Programming Documentation.

Annexure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64
#define MAX_PIPES 10

void display_prompt();
char* read_command();
char** parse_command(char *cmd, char *delim);
int count_pipes(char **args);
void execute_command(char **args);
void execute_with_redirection(char **args);
void execute_piped_commands(char **args);
int has_redirection(char **args);

int main() {
    char *input;
    char **args;

    while (1) {
        display_prompt();
        input = read_command();

        if (input == NULL) {
            printf("\n");
            break;
        }

        if (strlen(input) == 0) {
            free(input);
            continue;
        }
```

```
        args = parse_command(input, " \t\n");

        if (args[0] != NULL) {
            execute_command(args);
        }

        free(args);
        free(input);
    }

    return 0;
}

void display_prompt() {
    char *username = getenv("USER");
    if (username == NULL) username = "user";
    printf("[%s]$ ", username);
    fflush(stdout);
}

char* read_command() {
    char *input = malloc(MAX_INPUT);
    if (input == NULL) {
        perror("malloc");
        exit(1);
    }

    if (fgets(input, MAX_INPUT, stdin) == NULL) {
        free(input);
        return NULL;
    }

    size_t len = strlen(input);
    if (len > 0 && input[len-1] == '\n') {
        input[len-1] = '\0';
    }

    return input;
}
```

```
char** parse_command(char *cmd, char *delim) {
    char **args = malloc(MAX_ARGS * sizeof(char*));
    char *token;
    int i = 0;

    if (args == NULL) {
        perror("malloc");
        exit(1);
    }

    token = strtok(cmd, delim);
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, delim);
    }
    args[i] = NULL;

    return args;
}

int count_pipes(char **args) {
    int count = 0;
    for (int i = 0; args[i] != NULL; i++) {
        if (strcmp(args[i], "|") == 0) {
            count++;
        }
    }
    return count;
}

int has_redirection(char **args) {
    for (int i = 0; args[i] != NULL; i++) {
        if (strcmp(args[i], "<") == 0 || strcmp(args[i], ">") == 0) {
            return 1;
        }
    }
    return 0;
}
```

```
void execute_command(char **args) {
    if (count_pipes(args) > 0) {
        execute_piped_commands(args);
    } else if (has_redirection(args)) {
        execute_with_redirection(args);
    } else {
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            return;
        } else if (pid == 0) {
            execvp(args[0], args);
            perror(args[0]);
            exit(1);
        } else {
            waitpid(pid, NULL, 0);
        }
    }
}

void execute_with_redirection(char **args) {
    char *input_file = NULL;
    char *output_file = NULL;
    char *clean_args[MAX_ARGS];
    int j = 0;

    for (int i = 0; args[i] != NULL; i++) {
        if (strcmp(args[i], "<") == 0) {
            input_file = args[++i];
        } else if (strcmp(args[i], ">") == 0) {
            output_file = args[++i];
        } else {
            clean_args[j++] = args[i];
        }
    }
    clean_args[j] = NULL;

    pid_t pid = fork();
```

```
    if (pid < 0) {
        perror("fork");
        return;
    } else if (pid == 0) {
        if (input_file != NULL) {
            int fd_in = open(input_file, O_RDONLY);
            if (fd_in < 0) {
                perror(input_file);
                exit(1);
            }
            dup2(fd_in, STDIN_FILENO);
            close(fd_in);
        }

        if (output_file != NULL) {
            int fd_out = open(output_file, O_WRONLY | O_CREAT | O_TRUNC,
0644);
            if (fd_out < 0) {
                perror(output_file);
                exit(1);
            }
            dup2(fd_out, STDOUT_FILENO);
            close(fd_out);
        }

        execvp(clean_args[0], clean_args);
        perror(clean_args[0]);
        exit(1);
    } else {
        waitpid(pid, NULL, 0);
    }
}

void execute_piped_commands(char **args) {
    int num_pipes = count_pipes(args);
    char **commands[MAX_PIPES + 1];
    int cmd_idx = 0;
    int arg_idx = 0;

    commands[cmd_idx] = malloc(MAX_ARGS * sizeof(char*));
```

```
for (int i = 0; args[i] != NULL; i++) {
    if (strcmp(args[i], "|") == 0) {
        commands[cmd_idx][arg_idx] = NULL;
        cmd_idx++;
        arg_idx = 0;
        commands[cmd_idx] = malloc(MAX_ARGS * sizeof(char*));
    } else {
        commands[cmd_idx][arg_idx++] = args[i];
    }
}
commands[cmd_idx][arg_idx] = NULL;
int num_commands = cmd_idx + 1;

int pipes[MAX_PIPES][2];
for (int i = 0; i < num_pipes; i++) {
    if (pipe(pipes[i]) < 0) {
        perror("pipe");
        return;
    }
}

pid_t pids[MAX_PIPES + 1];

for (int i = 0; i < num_commands; i++) {
    pids[i] = fork();

    if (pids[i] < 0) {
        perror("fork");
        return;
    } else if (pids[i] == 0) {
        if (i > 0) {
            dup2(pipes[i-1][0], STDIN_FILENO);
        }

        if (i < num_commands - 1) {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        for (int j = 0; j < num_pipes; j++) {
```

```
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        exit(1);
    }
  }

  for (int i = 0; i < num_pipes; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
  }

  for (int i = 0; i < num_commands; i++) {
    waitpid(pids[i], NULL, 0);
  }

  for (int i = 0; i < num_commands; i++) {
    free(commands[i]);
  }
}
```